

The Prompt Ladder

Project

Build Your First CRUD API using FastAPI

Baseline Prompt

Help me build a CRUD API using FastAPI.

Output

The response explained what FastAPI is and suggested different project ideas like a task manager, library system, and notes app. It also asked questions before starting the implementation.

Notes

- **What changed in the prompt?**
This was my original prompt without any context.
 - **What improved in the output?**
It gave general guidance and suggested possible project ideas.
 - **What still failed?**
The response was too broad and didn't know my experience level or assignment requirements.
 - **What would I try next?**
Clearly explain what project I want to build.
-

Version 1 — Layer Added: Clear Goal

Prompt

Help me build a simple CRUD API using FastAPI for managing a task list.

Output

The response focused only on building a Task List API instead of suggesting multiple project ideas. It explained the basic project structure and CRUD operations.

Notes

- **What changed in the prompt?**
Added a clear project goal.
- **What improved in the output?**

The response became much more focused and directly related to my assignment.

- **What still failed?**

It still assumed I already understood backend concepts.

- **What would I try next?**

Mention that I am a beginner.

Version 2 — Layer Added: Audience

Prompt

Help me build a simple CRUD API using FastAPI for managing a task list. I am a beginner learning backend development.

Output

The explanation became much simpler. Instead of only giving code, it started explaining why each part of the code exists and what each library does.

Notes

- **What changed in the prompt?**

Specified that I am a beginner.

- **What improved in the output?**

The explanations became easier to understand and more beginner friendly.

- **What still failed?**

Some sections were still explained too quickly.

- **What would I try next?**

Ask the AI to explain everything step by step.

Version 3 — Layer Added: Context

Prompt

Help me build a simple CRUD API using FastAPI for managing a task list. I am a beginner learning backend development. This project is for my FlyRank internship assignment.

Output

The response started considering the assignment requirements. It suggested adding proper documentation, error handling, testing through Swagger UI, and explaining important design decisions that I could defend during evaluation.

Notes

- **What changed in the prompt?**
Added the real purpose of the project.
 - **What improved in the output?**
The response included suggestions related to the internship and documentation.
 - **What still failed?**
The explanation was still slightly advanced in some places.
 - **What would I try next?**
Request very simple language with no skipped steps.
-

Version 4 — Layer Added: Output Format

Prompt

Help me build a simple CRUD API using FastAPI for managing a task list. I am a beginner learning backend development. This project is for my FlyRank internship assignment.

Return:

1. Folder structure
2. Code
3. Explanation
4. Testing steps

Output

The response was neatly organized into sections. It first showed the project structure, then the code, followed by explanations and testing instructions.

Notes

- **What changed in the prompt?**
Requested a specific output format.
 - **What improved in the output?**
The information became much easier to follow because everything was properly organized.
 - **What still failed?**
Some explanations still assumed I already knew backend terminology.
 - **What would I try next?**
Ask for simple English and detailed explanations.
-

Version 5 — Layer Added: Constraints

Prompt

Help me build a simple CRUD API using FastAPI for managing a task list.

I am a beginner learning backend development.

This project is for my FlyRank internship assignment.

Return:

1. Folder structure
2. Code
3. Explanation
4. Testing steps

Use simple English.

Don't skip steps.

Assume I know only basic Python.

Output

The explanation became much more beginner friendly. Every concept was explained separately, including FastAPI, Uvicorn, Pydantic, CRUD operations, routing, testing with Swagger UI, and how to run the project from the terminal. The AI also explained why each piece of code was written instead of only showing the solution.

Notes

- **What changed in the prompt?**
Added learning constraints by asking for simple English and step-by-step explanations.
 - **What improved in the output?**
This version was the most useful. It explained everything in small steps and made it much easier for me to understand backend development instead of just copying the code.
 - **What still failed?**
The response became quite long because every concept was explained in detail. It took more time to read, but it helped me understand the topic better.
 - **What would I try next?**
If I build a larger backend project in the future, I would ask the AI to explain one feature at a time while keeping the same teaching style.
-

Final Reusable Prompt

Help me build a [PROJECT NAME] using [TECHNOLOGY].

I am a beginner learning this technology.

This project is for my internship assignment.
Teach me instead of simply giving the final solution.
Return your response in this order:

1. Folder structure
2. Complete code
3. Step-by-step explanation
4. Testing steps
5. Common mistakes to avoid

Use simple English.

Don't skip steps.

Explain why each piece of code is written.

Assume I know only the basics.

If possible, ask me questions occasionally so I understand the concepts instead of only copying the code.

What I Learned

Before this assignment, I used to write very short prompts and expected the AI to understand everything. Through this exercise, I realized that adding one improvement at a time makes a big difference in the quality of the response. Giving clear goals, audience, context, output format, and constraints helped me get explanations that matched my learning level. I also learned that a well-written prompt doesn't just produce better code—it produces better understanding.